

Laboratorio 2

Dispensa

Indice

1	Comandi base e primo programma C	3
1.1	Comandi base	3
1.2	Compilazione	3
1.3	Esempio di programma C	3
2	File, metadati e redirectione	4
2.1	<code>touch</code> e i metadati temporali	4
2.2	Copia e spostamento	4
2.2.1	Costo computazionale	4
2.3	Cancellazione	4
2.4	Visualizzare file	4
2.5	Wildcard (glob)	5
2.6	Input / Output	5
2.7	Pipe <code> </code>	5
2.8	Chi riceve gli argomenti?	5

1 Comandi base e primo programma C

1.1 Comandi base

Comando	Descrizione
<code>pwd</code>	<i>print working directory</i> : stampa in quale cartella sei
<code>ls</code>	elenca il contenuto della directory corrente
<code>ls -l</code>	elenco in formato lungo (permessi, proprietario, dimensione, data)
<code>cd</code>	<i>change directory</i> : navigo in una directory
<code>mkdir</code>	<i>make directory</i> : crea una directory
<code>echo \$?</code>	restituisce il valore di ritorno dell'ultimo comando eseguito

Percorsi speciali con `cd` :

- `.` indica la directory corrente
- `..` indica la directory padre

1.2 Compilazione

```
gcc -o first first.c # compila first.c e genera l'eseguibile first
gcc -Wall -Wextra -std=c11 -o first first.c # con tutti i warning e standard C11
```

1.3 Esempio di programma C

```
#include <stdio.h>

int main() {
    puts("hello"); // prende una stringa e va a capo
    printf("Risultato: %d\n", 5); // stringa formattata, non va a capo da sola
    return 0;
}
```

`printf` è una funzione a **numero variabile di parametri**: il primo è la stringa di formato, gli altri i valori da inserire al posto di `%d`, `%s`, ...

2 File, metadati e redirectione

2.1 touch e i metadati temporali

`touch` : se il file non esiste lo crea, se esiste ne aggiorna la data di modifica.

Metadato	Significato
<code>atime</code>	<i>access time</i> : ultimo accesso al file
<code>mtime</code>	<i>modification time</i> : ultima modifica del contenuto
<code>ctime</code>	<i>change time</i> : ultima modifica dei metadati

`touch` aggiorna `atime` e `mtime` al tempo corrente.

Aggiornare `atime` e `mtime` è già una modifica dei metadati, quindi normalmente cambia anche `ctime`.

`stat` : restituisce informazioni su un file, tra cui `atime`, `mtime`, `ctime`.

2.2 Copia e spostamento

`cp` : copia il contenuto di un file sorgente in un file destinazione.

- **Destinazione:** `mtime` e `ctime` aggiornati all'ora corrente (anche `atime` se il file viene creato). Con `-p`, `atime` e `mtime` vengono preservati dal sorgente (`ctime` no).
- **Sorgente:** `mtime` e `ctime` invariati; `atime` può essere aggiornato dalla lettura.

```
cp /tmp/ciao.txt ~/hello.txt # copia ciao.txt da /tmp nella home, rinominandolo
hello.txt
```

2.2.1 Costo computazionale

Operazione	Costo	Perché
Copia	$O(N)$	bisogna leggere tutti i byte del sorgente
Spostamento, stesso filesystem	$O(1)$	si aggiornano solo i metadati (il percorso)
Spostamento, altro filesystem	$O(N)$	copia nel nuovo filesystem + cancellazione dal vecchio

2.3 Cancellazione

Comando	Descrizione
<code>rm</code>	cancella un file
<code>rmdir</code>	cancella una directory vuota
<code>rm -r</code>	cancella una directory con tutto il suo contenuto (file e sottodirectory)

2.4 Visualizzare file

Comando	Descrizione
<code>cat</code>	stampa il contenuto di un file su stdout
<code>less</code>	come <code>cat</code> , ma permette di scorrere avanti/indietro e cercare
<code>head</code>	stampa le prime 10 righe

<code>tail</code>	stampa le ultime 10 righe
<code>head -n 5 file.txt</code>	stampa le prime 5 righe di <code>file.txt</code>

2.5 Wildcard (glob)

- `*` corrisponde a qualsiasi sequenza di caratteri. Es. `*.txt` = tutti i file che terminano in `.txt`.
- `?` corrisponde a **un solo** carattere qualsiasi. Es. `prova?.txt` corrisponde a `prova1.txt`, `provaA.txt`, ...

L'espansione è fatta **dalla shell**, non dal comando. Con `a.txt`, `b.txt`, `c.c` nella directory, `ls *.txt` diventa `ls a.txt b.txt`: `ls` non vede mai l'asterisco.

2.6 Input / Output

Redirigere lo standard output con `>` (sovrascrive o crea il file):

```
ls > elenco.txt
```

Aggiungere invece di sostituire con `>>`:

```
echo "prima riga" > log.txt
echo "seconda riga" >> log.txt
cat log.txt
# prima riga
# seconda riga
```

Redirigere lo standard input con `<` (legge da file invece che da tastiera):

```
programma < dati.txt
```

2.7 Pipe |

La pipe collega l'output di un comando all'input di un altro, senza passare per un file intermedio.

```
ls | less          # scorro l'elenco dei file con less
ls *.c | wc -l     # conta quanti file .c ci sono (wc -l conta le righe)
```

2.8 Chi riceve gli argomenti?

La shell separa la riga in parole e avvia il programma passandogli gli argomenti.

Vedremo come un programma C accede a questi argomenti tramite `argc` e `argv`, i parametri della funzione `main`.